UNIVERSITY OF PORTSMOUTH

School of Computing

M33147-INTELLIGENT DATA AND TEXT ANALYTICS

Coursework 2

Date of Submission : 19th January 2026

Module Lecturers : Dr Alaa Mohasseb

Dr Grace Golcarenarenji

Student ID(s) : up2541590

up2293411

Appendix A: Individual Contributions

Group Members:

1. **up2541590**

2. **up2293411**

Summary Table of Contributions

Group Member	Main Responsibilities	Specific Tasks Completed	Approx. % Contribution
up2541590	Text data preprocessing,Noise removal and data cleaning,Tokenization and lemmatization, Stop-word handling with negation preservation,Feature extraction using Bag-of-Words / TF-IDF	Performed text preprocessing including cleaning, tokenization, lemmatization, negation-aware stop-word handling, and feature extraction using Bag-of-Words and TF-IDF.	**50%**
up2293411	Sentiment classification, Machine learning model comparison, Performance evaluation metrics, BERT-based contextual modelling **Topic detection and topic quality assessment**	Ran analyses, created visualizations, trained models, documented findings clearly and finalized document	**50%**

Table of Contents

Text Preprocessing

These libraries are used to prepare raw text data for analysis. Pandas and NumPy handle data efficiently, while NLTK supports tokenization, stop-word removal, and lemmatization. Regular expressions and string tools clean unwanted characters, ensuring the text is structured, meaningful, and ready for machine learning models.

This line loads the IMDb review dataset into a structured table using pandas. The file is read as tab-separated data, where each row contains a movie review and its corresponding sentiment label. Naming the columns makes the data easier to process in later preprocessing and modelling steps.

This command displays the first few rows of the dataset to quickly understand its structure. It helps verify that the reviews and sentiment labels have been loaded correctly and allows an initial check for formatting issues before starting text cleaning and preprocessing.

	text	label
0	A very, very, very slow-moving, aimless movie ...	0
1	Not sure who was more lost - the flat characte...	0
2	Attempting artiness with black & white and cle...	0
3	Very little music or anything to speak of.	0
4	The best scene in the movie was when Gerardo i...	1

This command counts how many positive and negative reviews are present in the dataset. It helps understand the class distribution and checks whether the data is balanced, which is important before applying preprocessing and building classification models.

```
label
1    386
0    362
Name: count, dtype: int64
```

This command shows the total number of rows and columns in the dataset. It helps understand the size of the data, confirming how many reviews are available and how many features are present before starting text preprocessing and further analysis.

Exploratory Data Analysis (EDA)

This command provides a summary of the dataset, including column names, data types, and non-null values. It helps verify that the text and label columns are correctly loaded and checks for missing values before proceeding with text preprocessing and model building.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 748 entries, 0 to 747
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype 
---  ------  --------------  ----- 
 0   text    748 non-null    object
 1   label   748 non-null    int64 
dtypes: int64(1), object(1)
memory usage: 11.8+ KB
```

This step standardises the text data by converting all reviews to lowercase and removing punctuation. Lowercasing ensures that words with the same meaning are treated consistently, while removing punctuation eliminates unnecessary symbols that do not contribute to sentiment. Together, these steps reduce noise in the text and prepare the data for further cleaning and feature extraction.

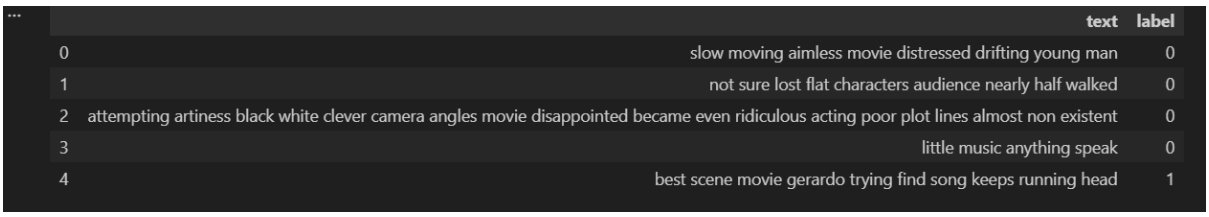
These checks are used to verify whether the preprocessing steps were applied correctly. The first line checks if contractions like “haven’t” still exist after cleaning. The second line searches for any remaining punctuation marks. The final line counts text entries that still contain numbers. Together, these checks help confirm that unwanted characters and patterns have been successfully removed from the text before moving to feature extraction and modelling.

```
text     59
label    59
dtype: int64
```

This step removes numerical values from the text data to reduce noise, as numbers do not usually contribute to sentiment meaning. The second line verifies that all digits have been successfully removed, ensuring the text is clean and suitable for further natural language processing tasks.

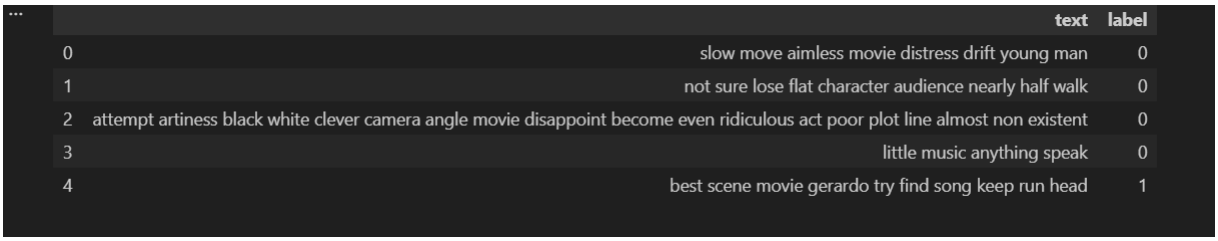
```
text     0
label    0
dtype: int64
```

This step removes common stop words from the text to reduce irrelevant information while carefully keeping important negation words such as “not” and “never”. Preserving these words helps maintain sentiment meaning. The final lines display the cleaned text clearly for verification before further processing.



	text	label
0	slow moving aimless movie distressed drifting young man	0
1	not sure lost flat characters audience nearly half walked	0
2	attempting artiness black white clever camera angles movie disappointed became even ridiculous acting poor plot lines almost non existent	0
3	little music anything speak	0
4	best scene movie gerardo trying find song keeps running head	1

This step applies lemmatization to convert words into their base form while considering their grammatical role. By using part-of-speech tagging, the lemmatizer produces more accurate root words, reducing vocabulary size and improving the quality of text representation for later classification tasks. It helps confirm that text cleaning, stop-word removal, and lemmatization have been applied correctly. Viewing the updated text ensures the data is clean, readable, and ready for feature extraction and modelling.



	text	label
0	slow move aimless movie distress drift young man	0
1	not sure lose flat character audience nearly half walk	0
2	attempt artiness black white clever camera angle movie disappoint become even ridiculous act poor plot line almost non existent	0
3	little music anything speak	0
4	best scene movie gerardo try find song keep run head	1

This step cleans up extra spaces created during earlier preprocessing steps. Multiple spaces are replaced with a single space, and leading or trailing spaces are removed. This ensures the text is neatly formatted and consistent, making it suitable for feature extraction and machine learning models.

Classification Using Bag-of-Words / Terms Representation.

This step prepares the data for text classification using a Bag-of-Words–based approach with TF-IDF weighting. The cleaned text is separated into features and sentiment labels, then split into training and testing sets to ensure fair evaluation. Stratified sampling is used to maintain class balance. The TF-IDF vectorizer converts text into numerical features by considering both single words and word pairs, while reducing the influence of very common terms. This representation captures important patterns in the text and serves as the input for machine learning classifiers used to compare performance across different algorithms.

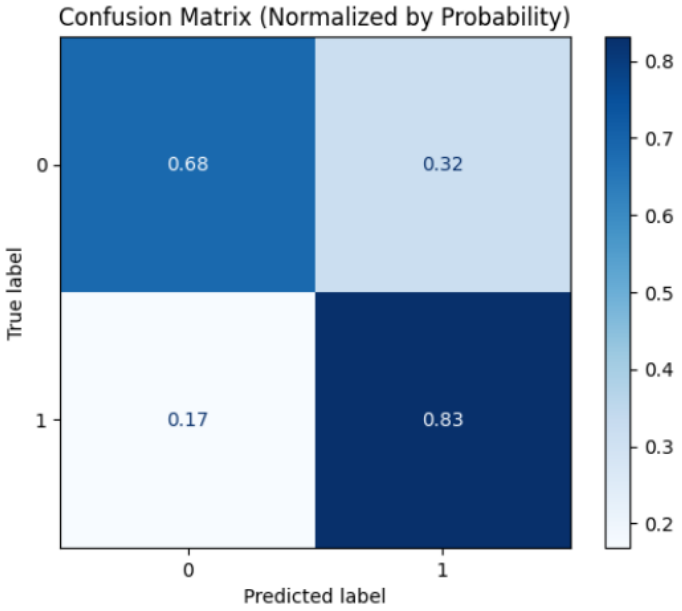
This step converts the training and testing text data into numerical TF-IDF feature vectors. The vectorizer is fitted only on the training data to avoid data leakage, then applied to the test data. Printing the shapes confirms the number of samples and features, ensuring the data is correctly prepared for classification models.

```
Train TF-IDF shape: (598, 7084)
Test TF-IDF shape:  (150, 7084)
```

This step trains a Logistic Regression classifier using TF-IDF features to predict sentiment labels. The model learns patterns from the training data and then generates predictions for the test set. A classification report is produced to evaluate performance using precision, recall, and F1-score. These metrics provide a detailed understanding of how well the model distinguishes between positive and negative reviews, supporting comparison with other classifiers.

```
              precision    recall  f1-score   support

           0       0.68      0.79      0.74        63
           1       0.83      0.74      0.78        87

    accuracy                           0.76       150
   macro avg       0.76      0.76      0.76       150
weighted avg       0.77      0.76      0.76       150
```

This step visualises the performance of the classification model using a normalised confusion matrix. It shows how accurately the model predicts each class by displaying true and false predictions as proportions, making it easier to interpret strengths and errors in sentiment classification.

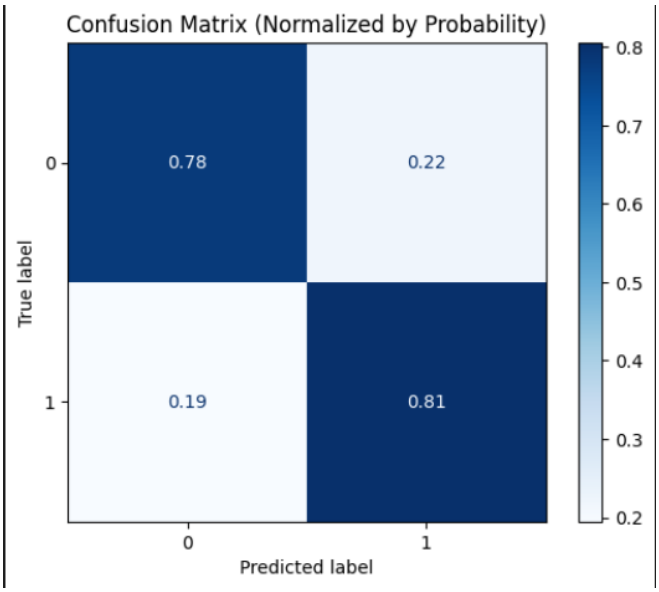


This step applies a Support Vector Machine (SVM) classifier to the TF-IDF features for sentiment classification. The model is trained on the training data with tuned hyperparameters to improve performance. Predictions are then made on the test set, and a classification report is generated to evaluate precision, recall, and F1-score. These results are later compared with other algorithms to identify the most effective model.

```
...                  precision    recall  f1-score   support

           0       0.78      0.79      0.79        72
           1       0.81      0.79      0.80        78

    accuracy                           0.79       150
   macro avg       0.79      0.79      0.79       150
weighted avg       0.79      0.79      0.79       150
```

This step uses GridSearchCV to optimise the SVM model by testing different values of hyperparameters. Cross-validation is applied to select the best combination based on ROC-AUC score, helping improve model performance and ensuring a fair and reliable comparison with other classifiers.

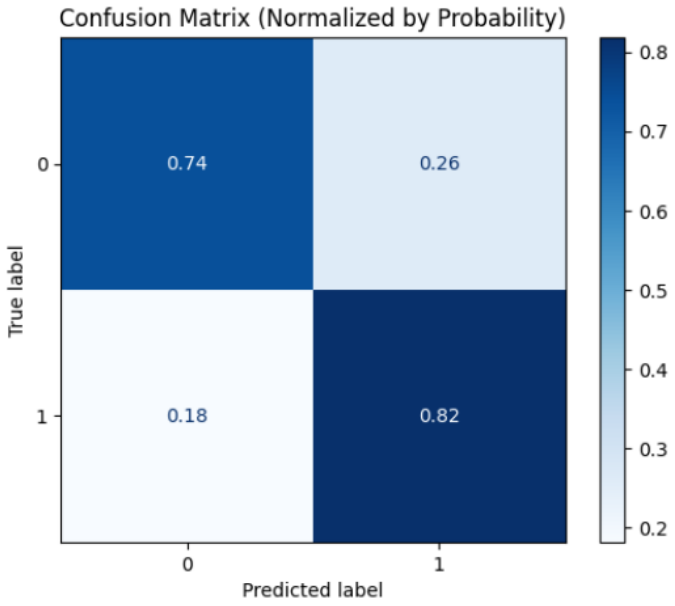
This step displays a normalised confusion matrix for the SVM classifier. It visualises the proportion of correct and incorrect predictions for each sentiment class, making it easier to evaluate model accuracy, identify misclassifications, and compare SVM performance with other classification algorithms.



This step implements a Naive Bayes classifier for sentiment classification using TF-IDF features. The model is trained on the training dataset and then used to predict labels for the test data. A classification report is generated to evaluate performance through precision, recall, and F1-score. Naive Bayes is particularly effective for text data due to its simplicity and efficiency, making it a useful baseline model for comparison with more complex classifiers.

	precision	recall	f1-score	support
0	0.79	0.74	0.77	73
1	0.77	0.82	0.79	77
accuracy			0.78	150
macro avg	0.78	0.78	0.78	150
weighted avg	0.78	0.78	0.78	150

This step visualises the Naive Bayes model's performance using a normalised confusion matrix. It shows the proportion of correct and incorrect predictions for each sentiment class, helping to identify classification errors and compare Naive Bayes results with Logistic Regression and SVM models.



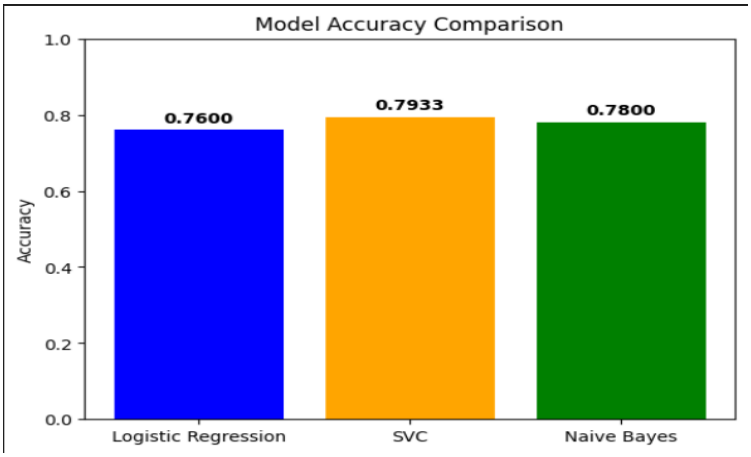
This step interprets the Logistic Regression model by identifying the most influential words for sentiment prediction. By examining model coefficients, the code extracts terms that strongly contribute to positive and negative classifications. Positive coefficients indicate words associated with positive sentiment, while negative coefficients indicate negative sentiment. This analysis improves model transparency and helps understand how specific words influence classification decisions, supporting a meaningful comparison of algorithm behaviour.

```
...
    Top Positive Features:
    give                    1.0349
    well                    0.9722
    good                    0.9462
    great                   0.9005
    enjoy                   0.8316
    excellent               0.8271
    wonderful               0.7764
    film                    0.7658
    funny                   0.7657
    brilliant               0.7442
    play                    0.7218
    character               0.6764
    beautiful               0.6029
    think                   0.5992
    love                    0.5797

    Top Negative Features:
    bad                     -2.7617
    awful                   -1.1743
    not                     -1.1392
    script                  -0.9816
    nothing                 -0.9348
    even                    -0.9309
    ...
    avoid                   -0.6695
    terrible                -0.6361
    slow                    -0.6168
    boring                  -0.6107
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

This step compares the performance of the three classification models using accuracy as a common evaluation metric. By calculating and printing accuracy scores for Logistic Regression, Support Vector Classifier, and Naive Bayes, it becomes easy to identify which algorithm performs best on the test data and supports a clear, fair comparison of results.

```
Logistic Regression Accuracy: 0.7600
SVC Accuracy: 0.7933
Naive Bayes Accuracy: 0.7800
```

This step visually compares the performance of the three classification models using a bar chart. Each bar represents the accuracy of Logistic Regression, SVM, and Naive Bayes, making differences in performance easy to understand. Displaying accuracy values above the bars improves clarity and supports a clear discussion of which model performs best for sentiment classification.



Classification (Error Analysis / Result Inspection)

BERT-Based Sentiment Analysis

A BERT-based model was applied to perform sentiment analysis and its performance was compared with traditional machine learning models. Unlike bag-of-words or TF-IDF approaches, BERT is a transformer-based model that understands the context and order of words within a sentence. This allows it to capture subtle language patterns such as negation, sarcasm, and word relationships more effectively. By fine-tuning the pre-trained BERT model on the IMDb dataset, the model was able to adapt its language understanding to the sentiment classification task. The results demonstrate how contextual models like BERT can provide improved performance and deeper linguistic understanding compared to conventional text classification methods.

The dataset was initially divided into training and testing sets using an 80/20 split. Stratified sampling was applied to preserve the same balance of positive and negative reviews in both sets. This approach

helps ensure a fair and reliable evaluation by allowing the models to be tested on data that accurately represents the overall dataset.

A pre-trained transformer model, **DistilBERT (distilbert-base-uncased)**, was chosen for this task. DistilBERT is a lightweight and efficient version of BERT that retains strong language understanding while requiring less computational power. This makes it well suited for faster training and limited hardware environments. To adapt the model for sentiment analysis, a classification head was added with two output labels representing positive and negative sentiment.

```
from transformers import AutoModelForSequenceClassification

model_name = "distilbert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(model_name)

model = AutoModelForSequenceClassification.from_pretrained(
    model_name,
    num_labels=2
)
```

Before training, the text reviews were tokenized using the BERT tokenizer, which converts sentences into numerical token IDs that the model can process. This tokenizer preserves contextual meaning and effectively handles unknown or rare words. The model was then fine-tuned on the training dataset for one epoch using a batch size of four. Fine-tuning enables the pre-trained model to adapt its internal parameters specifically for sentiment classification. This process is essential, as using BERT without fine-tuning would not allow the model to learn task-specific patterns or meet the assignment requirement.

```
training_args = TrainingArguments(
    output_dir="task3_bert_outputs",
    num_train_epochs=1,
    per_device_train_batch_size=4,
    per_device_eval_batch_size=4,

    logging_steps=20,
    report_to="none"
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_tok,
    tokenizer=tokenizer
)
```

After the training process, the fine-tuned model was used to generate sentiment predictions for the test dataset. The performance of the model was then evaluated using accuracy as the primary metric. In addition, the predicted labels were retained to support further error analysis, allowing a closer examination of incorrect predictions and overall model behaviour.

```
accuracy_score(y_test, y_pred)
✓ 0.0s
0.76
```

Error Analysis / Result Inspection

His step creates a results table to analyse the classification output in detail. It combines the original test reviews with their true labels, predicted labels, and a correctness flag. This allows manual inspection of correct and incorrect predictions, helping to understand model behaviour and common errors.

	text	true_label	predicted_label	correct
0	wonderful inspiring watch hope get release video dvd	1	1	True
1	anyway plot flow smoothly male bond scene hoot	1	0	False
2	unless visually collect extant film austen work skip one	0	1	False
3	love really scary	1	1	True
4	one fail create real suspense	0	1	False
...	...	...	...	...
145	no plot whatsoever	0	0	True
146	not good	0	1	False
147	word embarrassing	0	0	True
148	easily none cartoon make laugh tender way get dark sitcom orient teenager	1	0	False
149	felt though go ireland absolutely nothing whatsoever	0	0	True

150 rows × 4 columns

This step filters the test results to show only the incorrectly classified reviews. By counting these errors, it becomes easier to assess how often the model makes mistakes and to support a deeper discussion of classification performance and limitations.

```
(36, 4)
```

This step prints the total number of test samples and the number of incorrect predictions made by the model. It provides a clear summary of model performance and helps quantify classification errors, supporting a more detailed evaluation of results.

```
Total test samples: 150
Total errors: 36
```

This step separates misclassified reviews into false positives and false negatives. False positives occur when negative reviews are predicted as positive, while false negatives occur when positive reviews are

predicted as negative. Analysing these errors helps understand specific weaknesses of the model and supports a more detailed comparison of classification performance.

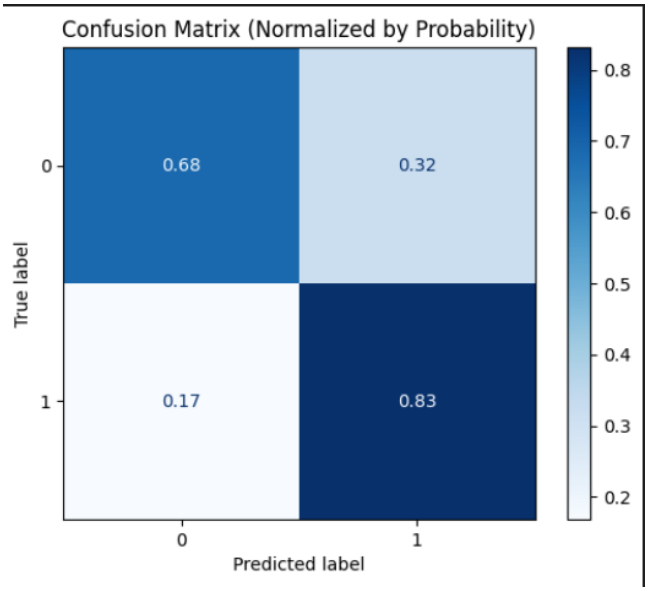
```
...   False Positives: 23
      False Negatives: 13
```

This step prints sample reviews that were incorrectly classified by the model. By viewing examples of false positives and false negatives, it becomes easier to understand why the model made certain mistakes. This qualitative analysis highlights challenges such as sarcasm or complex language and supports a deeper discussion of model limitations.

```
...   False Positives Examples:

      - unless visually collect extant film austen work skip one
      - one fail create real suspense
      - direct cinematography quite good

      False Negatives Examples:

      - anyway plot flow smoothly male bond scene hoot
      - barely boring moment film plenty humorous part
      - forget movie
```

This step performs a final evaluation of the classification results using multiple metrics. Accuracy provides an overall performance score, while the normalised confusion matrix visually shows how well each sentiment class is predicted. The classification report gives detailed measures such as precision, recall, and F1-score. Together, these outputs allow a clear and comprehensive assessment of the model's effectiveness and support meaningful comparison between different classification algorithms used.

```
...
      Accuracy: 0.76
```




```
Classification Report:
              precision    recall  f1-score   support

           0       0.79      0.68      0.74        73
           1       0.74      0.83      0.78        77

    accuracy                           0.76       150
   macro avg       0.76      0.76      0.76       150
weighted avg       0.76      0.76      0.76       150
```

Topic Detection / Topic Modelling

These libraries are used for topic modelling. CountVectorizer converts the cleaned text into a word-count matrix, while Latent Dirichlet Allocation (LDA) identifies hidden topics within the dataset. This task focuses on discovering common themes in the reviews rather than predicting sentiment.

This step converts the preprocessed text into a document–term matrix using word counts. The CountVectorizer limits very common words and removes English stop words to improve topic quality. Printing the matrix shape confirms the number of documents and unique terms, ensuring the data is correctly prepared for applying the topic modelling algorithm.

```
Document-Term Matrix shape: (748, 2403)
```

This step trains a Latent Dirichlet Allocation model to discover hidden topics within the reviews. The number of topics is set to ten as required by the task. Once fitted, the model learns patterns of word co-occurrence, allowing meaningful themes to be extracted from the dataset.

```
LDA trained with 10 topics.
```

This step is used to interpret and present the topics generated by the LDA model. After training, LDA stores word distributions for each topic, which indicate how strongly each word contributes to a specific topic. By extracting the feature names from the vectorizer and sorting the topic weights, the code identifies the top ten most relevant words for each topic. These words provide a clear representation of the underlying theme captured by each topic, such as opinions about acting, storyline, emotions, or overall movie quality. Printing the topics in this structured way makes the results easy to analyse and discuss. It also allows an assessment of topic coherence, helping determine whether the discovered topics are meaningful and distinct. This step directly supports the evaluation of topic quality required.

```
Topic 1:
movie, film, think, awful, recommend, like, bad, really, right, look

Topic 2:
movie, watch, say, think, rent, episode, work, mean, rating, plot

Topic 3:
film, watch, movie, bad, look, time, actor, excellent, play, boring

Topic 4:
movie, write, like, act, bad, look, come, scamp, film, beautiful

Topic 5:
bad, act, movie, character, know, make, good, think, really, write

Topic 6:
film, understand, great, waste, music, little, scene, know, masterpiece, cast

Topic 7:
movie, make, bad, really, thing, film, character, funny, work, minute

Topic 8:
movie, plot, love, film, character, way, worth, real, time, enjoy
...
film, movie, good, make, great, bad, like, character, act, line

Topic 10:
movie, film, time, like, script, character, story, bad, suck, actor
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

This step assigns a dominant topic to each document based on the probabilities generated by the LDA model. The transform method returns the likelihood of each review belonging to each of the ten topics. By selecting the topic with the highest probability, a single dominant topic is identified for every document. This information is then added to the dataset as a new column, making it easier to analyse how reviews are distributed across topics. Displaying sample rows helps verify that topic assignments are correctly applied. This step supports a deeper discussion of topic relevance, distribution, and quality, which is an important requirement.

	text	dominant_topic
0	slow move aimless movie distress drift young man	8
1	not sure lose flat character audience nearly half walk	0
2	attempt artiness black white clever camera angle movie disappoint become even ridiculous act poor plot line almost non existent	4
3	little music anything speak	5
4	best scene movie gerardo try find song keep run head	2
5	rest movie lack art charm meaning emptiness work guess empty	8
6	waste two hour	9
7	saw movie today think good effort good message kid	0
8	bit predictable	9
9	love cast jimmy buffet science teacher	7

This step helps interpret the discovered topics by displaying example reviews assigned to each topic. For every topic, a small number of sample documents are selected based on their dominant topic label. Reading these examples allows a qualitative assessment of whether the topic words identified earlier truly represent a coherent theme. If no documents are assigned to a topic, it indicates overlap between topics or limitations caused by a small dataset. This analysis is important for evaluating topic quality, interpretability, and relevance. By combining topic keywords with real review examples, this step strengthens the discussion and justification of the topic modelling results required.

```
================================================================================
TOPIC 0 — Sample documents
================================================================================
- not sure lose flat character audience nearly half walk
- saw movie today think good effort good message kid
- right case movie delivers everything almost right face

================================================================================
TOPIC 1 — Sample documents
================================================================================
- generally line plot weak average episode
- advise anyone go see
- several moment movie need excruciatingly slow move

================================================================================
TOPIC 2 — Sample documents
================================================================================
- best scene movie gerardo try find song keep run head
- give one look
- suspense feel frustration retard girl

================================================================================
TOPIC 3 — Sample documents
================================================================================
...
================================================================================
- waste two hour
- bit predictable
- cool
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

This step evaluates the quality of the topics by checking how much they overlap with each other. For each topic, the top words identified by the LDA model are stored as a set. The code then compares every pair of topics and counts how many top words they share. If two topics have a high overlap, it suggests that they may not be clearly distinct and could be capturing similar themes. Highlighting overlaps helps assess topic separation and coherence, which are important criteria in topic modelling. This analysis supports a critical discussion of the strengths and limitations of the detected topics and demonstrates a deeper evaluation of topic quality as required.

```
...   Topic 1 and Topic 3 overlap by 4 top-words.
      Topic 1 and Topic 4 overlap by 5 top-words.
      Topic 1 and Topic 5 overlap by 4 top-words.
      Topic 1 and Topic 7 overlap by 4 top-words.
      Topic 1 and Topic 9 overlap by 4 top-words.
      Topic 1 and Topic 10 overlap by 4 top-words.
      Topic 3 and Topic 4 overlap by 4 top-words.
      Topic 3 and Topic 7 overlap by 3 top-words.
      Topic 3 and Topic 8 overlap by 3 top-words.
      Topic 3 and Topic 9 overlap by 3 top-words.
      Topic 3 and Topic 10 overlap by 5 top-words.
      Topic 4 and Topic 5 overlap by 4 top-words.
      Topic 4 and Topic 7 overlap by 3 top-words.
      Topic 4 and Topic 9 overlap by 5 top-words.
      Topic 4 and Topic 10 overlap by 4 top-words.
      Topic 5 and Topic 7 overlap by 5 top-words.
      Topic 5 and Topic 9 overlap by 6 top-words.
      Topic 5 and Topic 10 overlap by 3 top-words.
      Topic 7 and Topic 8 overlap by 3 top-words.
      Topic 7 and Topic 9 overlap by 5 top-words.
      Topic 7 and Topic 10 overlap by 4 top-words.
      Topic 8 and Topic 9 overlap by 3 top-words.
      Topic 8 and Topic 10 overlap by 4 top-words.
      Topic 9 and Topic 10 overlap by 5 top-words.
```

REFERENCES

1. Intelligent Data And Text Analytics Module https://moodle.port.ac.uk/course/view.php?id=2317

2. Kaggle https://www.kaggle.com/

3. Pandas https://pandas.pydata.org/docs/user_guide/

4. Scikit Learn https://scikit-learn.org/stable/user_guide.html